Error-Tolerant E-Discovery Protocols*

Jinshuo Dong
Northwestern University
Evanston, IL, USA
jinshuo@northwestern.edu

Jason D. Hartline
Northwestern University
Evanston, IL, USA
hartline@northwestern.edu

Liren Shan
Toyota Technology Institute at Chicago
Chicago, IL, USA
lirenshan@ttic.edu

Aravindan Vijayaraghavan
Northwestern University
Evanston, IL, USA
aravindv@northwestern.edu

ABSTRACT

We consider the multi-party classification problem introduced by Dong, Hartline, and Vijayaraghavan (2022) in the context of electronic discovery (e-discovery). Based on a request for production from the requesting party, the responding party is required to provide documents that are responsive to the request except for those that are legally privileged[1]. Our goal is to find a protocol that verifies that the responding party sends almost all responsive documents while minimizing the disclosure of non-responsive documents. We provide protocols in the challenging non-realizable setting, where the instance may not be perfectly separated by a linear classifier. We demonstrate empirically that our protocol successfully manages to find almost all relevant documents, while incurring only a small disclosure of non-responsive documents. We complement this with a theoretical analysis of our protocol in the single-dimensional setting, and other experiments on simulated data which suggest that the non-responsive disclosure incurred by our protocol may be unavoidable.

CCS CONCEPTS

• **Applied computing** → **Law**; • **Theory of computation** → **Interactive proof systems**.

KEYWORDS

Classification; Multi-party Protocol; E-Discovery

ACM Reference Format:
Jinshuo Dong, Jason D. Hartline, Liren Shan, and Aravindan Vijayaraghavan. 2024. Error-Tolerant E-Discovery Protocols. In *Symposium on Computer Science and Law (CSLAW '24), March 12–13, 2024, Boston, MA, USA.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3614407.3643703

*Equal Contribution. The full version is available at https://arxiv.org/abs/2401.17952.

[1]The legally privileged documents are protected by privilege, which allows the responding party to resist the mandatory disclosure and withhold such documents in the legal proceeding. Documents that are sensitive or confidential are not necessarily privileged. In this work, we focus on the e-discovery task that aims to find almost all responsive documents among those that are not privileged.

This work is licensed under a Creative Commons Attribution-NonCommercial International 4.0 License.
CSLAW '24, March 12–13, 2024, Boston, MA, USA
© 2024 Association for Computing Machinery.
ACM ISBN 979-8-4007-0333-1/24/03.
https://doi.org/10.1145/3614407.3643703

1 INTRODUCTION

In legal proceedings, a plaintiff (Bob) may need relevant evidence from a corpus of electronic documents possessed by a defendant (Alice). To acquire this evidence Bob initiates an electronic discovery (e-discovery) process by issuing a request for production to Alice to identify and provide the documents that are responsive to the request. A few main concerns of the e-discovery process are (a) the efficiency and accuracy by which it is conducted, e.g., Grossman and Cormack [2010]; (b) privacy of documents not responsive to the request [Stuart 2021]; and (c) accountability for an inaccurate or incomplete review, e.g., via sanctions to Alice's legal team, e.g., *Brown* v. *Tellermate Holdings Ltd.* [2014].

Technology-assisted review (TAR) tools are sometimes used in e-discovery processes to identify relevant documents among the massive collection of electronic documents. Grossman and Cormack [2010] show that TAR can accurately retrieve relevant documents with significantly less human effort in manual review. TAR tools have been approved for use in court cases, e.g., *Moore* v. *Groupe* [2012]. Kluttz and Mulligan [2019], however, argue that accountability is potentially a significant issue for these tools and may limit their adoption; moreover, courts have explicitly not obligated their use, e.g., *Hyles* v. *New York City* [2016]. Further discussion of the literature on TAR is in the related work section.

To address the accountability issue of TAR tools, Kluttz and Mulligan [2019] compares them to medical devices for which there is an approval board that decides which are suitable for use. They advocate the design of methods and standards for the validation and testing of these tools as the tools of medicine are reviewed by experts before their adoption. They also suggest developing contestable TAR tools that provide transparency, interaction, and configurability.

The starting point for this paper, relative to the above legal scholarship on technology assisted review, is the observation that generally algorithms (which underlie TAR tools) have affordances that other technologies (such as medical devices) do not possess. Specifically, in certain contexts (a) algorithms can be designed to produce proofs that they are correct in every instance in which they are run (a classical example of these algorithms are zero-knowledge proofs, Goldwasser et al. 1989), and (b) algorithms can be partitioned in to multi-party protocols where the computation is divided among the parties in a way that enables them to ensure that information

is private (a classical example of these algorithms are secure multi-party computations, Goldreich et al. 1987). A multi-party protocol is one where each party runs an algorithm and these algorithms interact to compute the desired outcome.

Algorithms are developed from theoretical foundations to practical applications through successive efforts of algorithms researchers. Dong et al. [2022] demonstrates the plausibility of developing multi-party protocols for e-discovery that satisfy the previous two concerns of proving their correctness (which gives accountability) and maintaining (some) privacy. They do so in a restricted scenario that is a frequent starting point for developing new machine learning algorithms: the data is represented by points in high-dimensional space, the responsive and non-responsive documents are perfectly separated by a linear function (a.k.a., classifier), and the hand-labeling procedure that Alice or Bob might use to determine which points are responsive and which are non-responsive has perfect accuracy.

Our work relaxes the impractical requirements of Dong et al. [2022] by giving an e-discovery protocol that finds a good linear classifier when no linear classifier perfectly labels the points and while allowing for the possibility that Alice makes some mistakes in labeling the points. (These mistakes when caught will be refuted by Bob and resolved in the court.) Grossman and Cormack [2010] and Grossman and Cormak [2012] demonstrate that even the best human reviewers may make mistakes in review tasks inadvertently, thus, dealing with mistakes is fundamental to bringing e-discovery protocols closer to practice. A final property of the protocol we develop is that it fits into the state-of-the-art Continuous Active Learning (CAL) paradigm of Cormack and Grossman [2014] and thus we can directly compare its performance to standard algorithms for technology assisted review (See Figure 1).

In classification where there is no classifier that makes no mistakes, known in machine learning as the *non-realizable* case, there is a tradeoff between minimizing classification errors that come from false positives and false negatives. In e-discovery false positives are documents that are incorrectly labeled as responsive when in fact they are non-responsive. In our analysis we will measure the false-positive rate by tracking this *non-responsive disclosure*, i.e., the number of non-responsive documents that are disclosed to Bob. Notice that our protocols will generally also disclose some documents that it correctly classifies as non-responsive in order to prove that its selected classifier is correct. These additionally disclosed documents will be included in our analysis of non-responsive disclosure. False negatives are responsive documents that are incorrectly labeled as non-responsive. In our analysis we will measure the false-negative rate as the *recall*, the fraction of responsive documents that are disclosed by the protocol. Note for contrast that in the realizable case for the optimal classifier, the non-responsive disclosure is zero and the recall is one.

In this language, Dong et al. [2022] studied the realizable setting and developed a protocol with recall of one, i.e., all positive documents are disclosed, that provably minimized the non-positive disclosure. In this case the non-responsive documents disclosed were correctly classified as non-responsive, but must be disclosed to prove that the selected classifier is correct. This non-responsive disclosure enables the protocol not to reply on the honesty of Alice's labeling of documents. The TAR algorithms in the literature, e.g., from Cormack and Grossman [2014], rely on the honesty of Alice's legal time and, though the classifier they identify may have false negatives, all documents that are actually disclosed to Bob undergo manual review and are labeled as responsive, thus, there is no non-responsive disclosure. Instead, they aim to tradeoff the number of documents that are reviewed by hand with the recall. There is a trivial method for making these protocols accountable, i.e., so as to not rely on Alice's honesty, which is to release to Bob all documents that are subject to manual review. In this case, the non-responsive disclosure will be equal to the number of non-responsive documents that are manually reviewed. We can therefore view existing TAR algorithms as identifying a baseline to which our accountable protocols can be compared. Our goal in this comparison is to have nearly as large a recall but far lower non-responsive disclosure.

Our contributions are summarized as follows

(1) We propose a *Label-Verification protocol* for the case where the documents are projected into a single-dimension and a linear classifier is simply a threshold above which all points are classified as responsive. This protocol detects with high probability the manipulation of a dishonest defendant (Alice), while only revealing a small number of non-responsive documents to the plaintiff (Bob). Our e-discovery protocol for the non-realizable setting is constructed by employing the Label-Verification protocol as a subroutine in the Continuous Active Learning (CAL) method proposed by [Cormack and Grossman 2014]. On the one hand, our protocols address the accountability concern in existing TAR methods. On the other hand, our protocols tackle the more challenging non-realizable setting that is not handled by the existing work of Dong et al. [2022]. We explain our protocols in Section 2.
(2) We demonstrate empirically on review task datasets from the TREC 2009 Legal Track Interactive Task [Cormack and Mojdeh 2009; Hedin et al. 2009] that compared to the baseline protocol Reveal_All which is CAL method by revealing all requested documents for accountability, our protocol incurs at most 10% decrease in recall (i.e., the fraction of discovered responsive documents among all responsive documents) with a significant improvement in terms of the protection of non-responsive documents (in comparison to a naive implementation which is accountable but does not private). Specifically on the above datasets, our protocol has only at most 10% lower recall, while the non-responsive disclosure is up to 75% smaller in comparison to the CAL method where all manually reviewed documents are disclosed. A preview of our results is included in Figure 1. The details and more results can be found in Section 3.
(3) On the theoretical front, we show that the (training) recall of our Label-Verification protocols are provably close to the optimal regardless of the manipulation from the defendant in the single-dimensional case. We also prove that if the defendant reports truthfully, then the non-responsive disclosure of our protocols is small. These results are stated in Theorems 4.1 and 4.3. This gives an explanation for the significant improvement in non-responsive disclosure curves in experiments, as the Label-Verification protocol forms a key subroutine in our e-discovery protocol (See Figure 1).

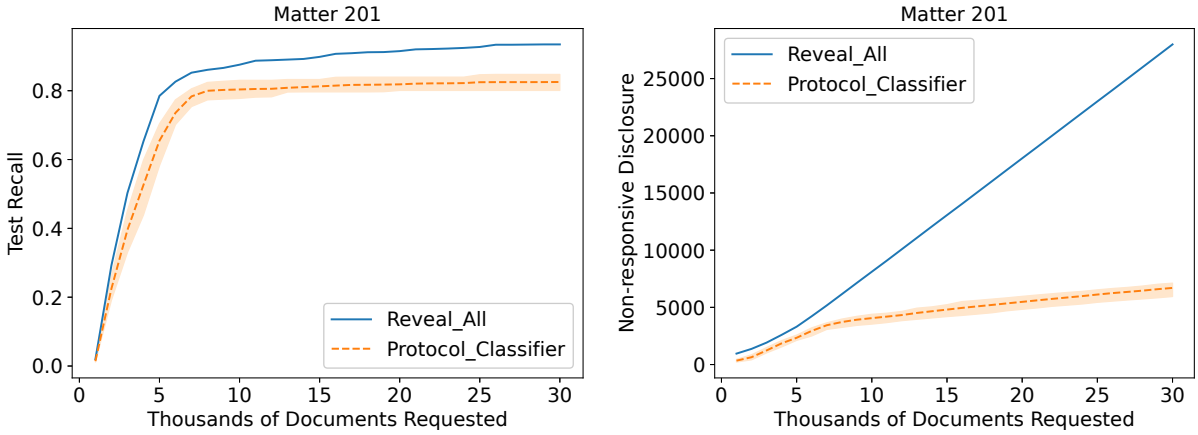

Figure 1: Compared is the Continuous Active Learning (CAL) method of Cormack and Grossman [2014] with two different internal labeling procedures. "Protocol_Classifier" uses the labeling protocol that we develop. "Reveal_All" provides the naive baseline of the labeling protocol where all documents that require hand labels are provided to the plaintiff. "Protocol_Classifier" and the baseline "Reveal_All" are both accountable CAL-based protocols, which means they do not rely on the honesty of the defendant. We compare the recall and non-responsive disclosure of these two CAL-based protocols. Note that the version of the CAL method which only reveals responsive documents labeled by hand is not accountable. Experimental results are given for the dataset Matter 201. The left-side figure shows the test recall of two protocols implementing the CAL method that requests $N = 1000$ new documents for review in each iteration as the number of iterations $T = 1, 2, \ldots, 30$. The right-side figure shows the non-responsive disclosure of two protocols as the number of iterations increases. We repeat our protocol ten times and plot the average recall and the non-responsive disclosure with the corresponding ranges.

We simulate the defendant with the truthful report in our experiments since it is hard to characterize the best response of the defendant without any assumption on her objective. With proper assumptions on the instance and the objective function of the defendant, we partially characterize the best response of the defendant to our label report protocol in Theorem 4.2. We show this best response is close to the truthful report and has similar guarantees. Note that in high dimensions, classic results on agnostic learning show that learning a linear classifier in the non-realizable setting is computationally challenging even without any accountability constraints (see the related work section below). Our theoretical results are elaborated in Section 4.

(4) Finally, we show empirically that the non-responsive disclosure is unlikely to have significant room of improvement. We show that such a loss is expected on the simulated data even in the realizable setting, where it is known that the critical point protocol incurs provably minimal non-responsive disclosure [Dong et al. 2022]. We observe that the fraction of revealed documents in all non-responsive documents are comparable for the two cases. This is made possible by an improved algorithmic implementation of the critical point protocol, which combines a new analysis from the perspective of projective geometry with a classical result from Clarkson [1994]. These results can be found in Section 5.

Related Work. This work studies the e-discovery problem with accountability considerations in the linear non-realizable setting. Without any accountability considerations, this is the classic problem of *agnostic* learning a halfspace [Kearns et al. 1992]. This is a notoriously challenging problem that is believed to be computationally hard in high dimensions see e.g., [Guruswami and Raghavendra 2009; Kalai et al. 2008; Kearns and Vazirani 1994]. Efficient algorithms are known only under additional assumptions e.g., log-concavity of the distribution generating the data points [Awasthi et al. 2017], which we do not assume in this work.

In the literature on TAR tools, the work most relevant to our work is the continuous active learning (CAL) method proposed by Cormack and Grossman [2014, 2017]. The CAL method iteratively improves the recall in the e-discovery task by interacting with the human reviewers. The CAL method achieves the highest recall in many document retrieval tasks. Therefore, active learning techniques are considered the state-of-the-art in the TAR tools [O'Mara-Eves et al. 2015; Zou and Kanoulas 2020]. Recent advances in natural language processing, especially large language models, provide new techniques for document retrieval tasks. [Chang et al. 2020] introduced a two-tower transformer model with pre-training for large-scale retrieval tasks. For legal tasks, Louis and Spanakis [2021] proposed a RoBERTa-based retrieval model (where RoBERTa is a transformer) on the Belgian Statutory Article Retrieval Dataset (BSARD). In these works, transformers provide more informative embedding of documents, which can potentially be applied in the CAL method and our protocols. We leave this as an interesting open direction.

The multi-party protocols have been widely used to achieve secure computation and preserve privacy in many applications.

Dong et al. [2022] proposed the e-discovery protocol for the realizable setting. Goldwasser et al. [2021] studies a different two-party classification problem called PAC verification, that is motivated by delegation of computation. Here the aim is for one party (prover) to convince the other party (verifier) about the correctness of a potential classifier with much fewer samples than it would take for the verifier to learn the classifier. The difference is that in PAC verification, both the prover and the verifier have access to the distribution of labeled examples. In contrast, only one party (defendant Alice) has access to the distribution of labeled examples in our setting.

2 MODEL

In this section, we give a high-level description of our e-discovery protocols. We first define the multi-party protocol model for the e-discovery task and measurements to evaluate such protocols. We then provide our e-discovery protocol based on the continuous active learning (CAL) method introduced by Cormack and Grossman [2014]. Despite its efficiency, the CAL method involves manual review of documents and hence is challenging to achieve accountability and privacy simultaneously. We propose the Label-Verification Protocol (Figure 3) to implement the review step accountably with a small disclosure of non-responsive documents. The overall protocol (Figure 2) then follows by using the Label-Verification Protocol as a subroutine in the CAL method. More details of our protocols can be found in Section 3.

Multi-party Protocol Model. We consider the design of the multi-party protocols as proposed by Dong et al. [2022]. There are three parties: Alice (defendant), Bob (plaintiff), and a trusted third party called Trent. Alice has a set of documents X. Bob has issued a request for production, which requires Alice to reveal the documents that are responsive to this request. Cloud computing service providers like Gmail and Outlook, or the court system can play the role of Trent. The function $f : X \to \{-1, +1\}$ denotes the true label of these documents regarding this request for production. For each document $x \in X$, it is responsive if $f(x) = 1$; otherwise, it is non-responsive. The words "label" and "responsiveness" will be used interchangeably in the rest of the paper. An instance (X, f) is a set of documents X with true labels f. Let $n = |X|$, $n^+ = |\{x \in X : f(x) = 1\}|$, and $n^- = |\{x \in X : f(x) = -1\}|$ be the number of data points, positive points, and negative points in X, respectively.

The multi-party protocol contains potentially multiple rounds of interaction between Alice, Bob, and Trent. Both Alice and Bob can label the documents. Trent has access to all documents but can not label them by himself. The outcome $B \subseteq X$ of a multi-party protocol is defined to be a subset of documents in X that are revealed to Bob.

We now define two measurements that we used to evaluate the multi-party protocols. The goal of the e-discovery protocol is to retrieve nearly all responsive documents reliably while minimizing the privacy loss of the defendant (Alice) caused by revealing non-responsive documents to the plaintiff (Bob). Thus, we measure the efficacy of the e-discovery protocol by the *recall*, a.k.a. true positive rate, which is the most common measurement of TAR tools used in the e-discovery literature. We measure the privacy loss of the defendant in the e-discovery protocol by the *non-responsive disclosure*, which is the number of false positives and negative documents revealed to ensure accountability. Thus, our goal is to design an e-discovery protocol that has a relatively large recall and a small non-responsive disclosure.

Definition 2.1. *Given an instance (X, f) and an e-discovery protocol M, let $B \subseteq X$ be the outcome of this protocol, which is the set of documents revealed to the plaintiff. The* recall *of this outcome is the fraction of responsive documents that are retrieved in B*

$$\mathrm{REC}(B) = \frac{|\{x \in X : x \in B, f(x) = +1\}|}{|\{x \in X : f(x) = +1\}|}. \tag{1}$$

The non-responsive disclosure *of this outcome B is the number of non-responsive documents revealed to the plaintiff*

$$\mathrm{NRD}(B) = |\{x \in X : x \in B, f(x) = -1\}|. \tag{2}$$

We now define the non-realizable setting and the benchmark for e-discovery protocols. As a natural language processing tool, the e-discovery protocol will first embed all documents into an Euclidean space with dimension d. With an abuse of notation, let $X = \{x_1, \ldots, x_n\} \subset \mathbb{R}^d$ be the embedding of all documents. For any two vectors $x, y \in \mathbb{R}^d$, we use $x \cdot y$ to denote the inner product of these two vectors. We consider the linear classification on this embedded instance (X, f). A linear classifier $h : X \to \{-1, +1\}$ partitions the d-dimensional space into two parts with a hyperplane $w \cdot x + b = 0$ where $w \in \mathbb{R}^d$ and $b \in \mathbb{R}$. The vector w is the normal vector of this hyperplane. For each document $x \in X$, $h(x) = 1$ if $w \cdot x + b \geq 0$; otherwise, $h(x) = -1$. The error of a linear classifier h on the embedded instance (X, f) is defined to be the number of misclassified documents $\mathrm{err}(h) = |\{x \in X : h(x) \neq f(x)\}|$. Let h^* be the optimal linear classifier for the instance (X, f) with the minimum error $\mathrm{err}^* = \mathrm{err}(h^*)$.

An embedded instance (X, f) is realizable (or linearly separable) if and only if the optimal classifier h^* perfectly classifies all documents, i.e. $\mathrm{err}^* = 0$. Otherwise, this instance (X, f) is called non-realizable, which implies the error of the optimal linear classifier is strictly greater than 0. Let $\mathrm{FN}(h) = |\{x \in X : f(x) = 1, h(x) = -1\}|$ be the false negative error of a linear classifier h. Let $\mathrm{FN}^* = \mathrm{FN}(h^*)$ be the false negatives of the optimal linear classifier. In the non-realizable setting, revealing $B^* = \{x \in X : h^*(x) = 1\}$ according to the optimal linear classifier achieves a recall $\mathrm{REC}^* = \mathrm{REC}(B^*) = 1 - \mathrm{FN}^*/n^+ < 1$. Thus, the minimum error err^* and this recall REC^* given by h^* reflect the inherent difficulty of this instance. We use the optimal linear classifier and its recall REC^* as our benchmark and aim to design a protocol that achieves a recall close to REC^* with a small non-responsive disclosure.

The CAL Method. To find responsive documents efficiently, Cormack and Grossman [2014] proposed a technology-assisted review (TAR) method, continuous active learning (CAL). The CAL method first embeds all documents into a high-dimensional Euclidean space $\mathbb{R}^d$. The CAL method iteratively maintains and grows a subset of documents $S \subset X$ (with random initialization) which are labeled by human reviewers. In each iteration, the classifier is first updated by training a linear support vector machine (SVM) on the current S. Each remaining document is assigned a score based on the SVM classifier, which is the projection to the normal vector w of the hyperplane corresponding to the SVM classifier. Then the

N documents with the highest scores among the remaining documents are actively chosen and manually reviewed to confirm their labels. The score of each of the remaining documents represents how likely this document is to be labeled responsive according to the updated classifier. Thus, these N documents are the most likely to be labeled responsive in the remaining unreviewed documents based on the updated classifier. After the manual review, these N labeled documents are added to S and this finishes one iteration. The pseudo-code is summarized in Algorithm 1.

Algorithm 1 Continuous Active Learning (CAL)

1: **Input:** a collection of documents X, a document review query, parameters T and N, where T is the number of iterations and N is the number of requested documents in each iteration.
2: **Output:** a set of responsive documents
3: Let set $S = \varnothing$.
4: Select N documents randomly from the results of a seed query search.
5: **for** $i = 1, 2, \cdots, T$ **do**
6: *Manually review* N new documents and add them with their labels to set S.
7: Train a linear classifier with the support vector machine on dataset S.
8: Compute the score of each remaining document as the projection to the normal vector w of the hyperplane corresponding to the current linear classifier.
9: Select N documents from $X \setminus S$ with the highest scores.
10: **end for**
11: **return** responsive documents in S.

E-discovery Protocol. We now describe our e-discovery protocol, which implements the CAL method reliably with a verification process. The CAL method is an e-discovery algorithm that makes repeated calls to a primitive e-discovery algorithm (which is the manual review in the original CAL). When that primitive e-discovery algorithm is a multi-party protocol (as defined previously), then the combination is a multi-party protocol as well.

Note that the CAL method requires the manual review of new documents added in each iteration (Step 9 of Algorithm 1), which causes accountability concerns. Alice may intentionally hide responsive documents in this manual review process. To implement this step reliably and accountably, in our protocol, Trent simulates the CAL method by calling on Alice and Bob in the Label-Verification protocol. The framework of our e-discovery protocol is shown in Figure 2. In each iteration, Trent computes a set of N new documents as in Step 9 of Algorithm 1. Let R be this set of documents embedded in the single-dimensional space $\mathbb{R}$ according to their scores computed in Step 8 of Algorithm 1. Then, Trent call the Label-Verification protocol to retrieve responsive documents in this sub-instance (R, f).

Label-Verification Protocol. We now describe the main subroutine of our protocol, the single-dimensional Label-Verification protocol. The Label-Verification protocol is a multi-party protocol, which aims to retrieve responsive documents in the input instance (R, f) accountably. We first introduce a high-level, "template" protocol,

(1) Input the set of all documents X and parameters T, N of the CAL method.
(2) Trent simulates the CAL method on X with T iterations.
(3) In each iteration $i \leq [T]$, let $R \subseteq X \setminus S$ be the set of N new documents selected in Step 9 of Algorithm 1.
(4) Trent runs the Label-Verification protocol with Alice and Bob on the sub-instance (R, f), where f is the true label function and documents in R are embedded in $\mathbb{R}$ with their scores computed in Step 8 of Algorithm 1.
(5) Trent adds these documents R and their labels returned by the Label-Verification protocol to set S and start a new iteration of the CAL method.
(6) Output the responsive documents in S, which are revealed to Bob.

Figure 2: E-discovery Protocol

which covers protocols for non-realizable data proposed in this work. We then give the specific implementations.

(1) Input a set of documents R embedded in the single-dimensional space $\mathbb{R}$
(2) Trent requests Alice to report the labels of documents in R.
(3) Trent iteratively selects documents in R and reveals those documents to Bob.
(4) Bob labels received documents and reports the labels to Trent.
(5) Trent checks the agreement of reports from Alice and Bob. If Bob disagrees with Alice's label for a document, then this document is sent to the court to settle.
(6) Trent checks the stopping condition and decides to stop or go to Step 2 (send more documents to Bob).
(7) Output the labels of all documents in R as follows. Each document is labeled by the court decision if this document is sent to the court; otherwise, it is labeled by Alice's report.

Figure 3: Label-Verification Protocol

The template label-verification protocol we propose is the following iterative process. It first requests Alice to report the labels of all documents in R. Then, it iteratively sends some documents in R to Bob for verification until the stopping condition is satisfied. If Bob disagrees with Alice on the label of a document, then this document is sent to the court to settle. If the court disagrees with Alice on the label of this document, then we say Alice made an error on this document. The template Label-Verification protocol is shown in Figure 3.

There is a trivial Label-Verification protocol, which reveals all documents in R to Bob for verification in Step 3 of Figure 3. This trivial protocol has perfect accountability (recall) on the hand-labeled documents R since Bob receives all responsive documents in R. While this protocol also reveals all non-responsive documents in R to Bob. Thus, it has the maximum non-responsive disclosure on the sub-instance (R, f). We use the CAL with this trivial protocol as the baseline, denoted as Reveal_All.

Two details need to be specified in order to recover the concrete protocols in our work: (1) how Trent selects the subset of documents in Step 3; and (2) the stopping condition in Step 6. In the non-realizable case, the following specifications yield our two Label-Verification protocols. Further details can be found in Section 4

- Selection: Trent first computes the optimal classifier threshold $t_A \in \mathbb{R}$ for Alice's report. Then, Trent sends all documents in $[t_A, \infty)$ to Bob for verification. If t_A is the optimal classifier for the true label, then the recall is already the recall REC^* of the optimal classifier. If t_A is not the optimal classifier for the true label, then Alice hides positive points close to t_A as negative points. To detect this case, points in $(\infty, t_A]$ reported as negative by Alice are sampled independently in the decreasing order of their positions. Generally, the points closer to t_A are sampled earlier with higher probabilities. The specific probability depends on the honesty of Alice's prior reports. These sampled points are revealed to Bob.
- Stopping condition: The Label-Verification protocol stops if enough errors are detected in Alice's report, or when no more documents can be sampled (If a document is not sampled in the first time, then this document can not be sampled again.). In Step 5, this document is sent to the court to settle if Bob disagrees with Alice's label for this document. If a document is sent to the court, then the protocol outputs the court decision as the label of this document. Otherwise, it labels this document by Alice's report. Note that all points labeled as positive in the output are already verified by both Alice and Bob. We say Alice made an error on the label of a document if and only if the court disagrees with Alice's label.

3 EMPIRICAL RESULTS

In this section, we show the empirical results of our protocols. Our protocols are constructed by combining our Label-Verification protocols in Section 4 with the CAL method shown in Algorithm 1.

We use the review task, Matters 201 and 202 from the TREC 2009 Legal Track Interactive Task [Cormack and Mojdeh 2009; Hedin et al. 2009]. Matter 201 is the requests for production of all documents related to the Company's engagement in prepay transactions. Matter 202 is the requests for production of all documents related to the Company's engagement in transactions characterized as compliant with FAS 140 or FAS 125. The information of datasets is provided in Table 1, including the number of total documents and the number of responsive documents for each review task. We refer to Cormack and Grossman [2014] for a more detailed description of these review tasks.

Matter	Number of Documents	Number of Responsives
201	723537	2154
202	723537	8734

Table 1: Information of datasets, Matters 201 and 202 from the TREC 2009 Legal Track Interactive Task.

Our protocols implement the CAL method (Algorithm 1) with parameters $N = 1000$ and $T = 1, 2, \cdots, 30$. The CAL method is used as a TAR tool to find responsive documents by iteratively selecting new documents for manual review. Note that this method requires labeling the 1000 documents that are selected in each iteration. We consider the multi-party implementation of the CAL method. Trent implements the CAL method by using a Label-Verification protocol to label documents in each iteration. In the experiments, we implement the following three protocols:

- Reveal_All: A naive Label-Verification protocol is to reveal all 1000 documents to Bob for verification. We use Reveal_All to denote the e-discovery protocol by combining the CAL method with this naive protocol. This protocol has the same recall as the CAL method, but might incur a large non-responsive disclosure. We use this protocol as our baseline.
- Protocol_Label: We use the sampling protocol for label report in Section 4.1 (Algorithm 2) as the Label-Verification protocol. Protocol_Label is the e-discovery protocol by combining the CAL method with this Label-Verification protocol. We set parameters $\delta = 0.01$ and $k = 1$ in the sampling protocol for label report. We simulate Protocol_Label by assuming Alice truthfully reports all true labels of documents.
- Protocol_Classifier: We use the sampling protocol for classifier report in Section 4.2 (Algorithm 3) as the Label-Verification protocol. Protocol_Classifier is the e-discovery protocol by combining the CAL method with this Label-Verification protocol. We set parameter $\delta = 0.01$ in the sampling protocol for classifier report. We simulate Protocol_Classifier by assuming Alice truthfully reports the optimal classifier.

Results. We now describe our empirical results for recall and non-responsive disclosure of these three protocols. Figure 4 shows the test recall of three protocols as the number of documents that Trent requests Alice to review increases. Since these three protocols are based on the CAL method, the number of hand-labeled documents increases by $N = 1000$ for each iteration as the number of iterations used in the CAL method increases. Since the baseline Reveal_All reveals all hand-labeled documents to Bob for verification, it achieves full accountability on the label of hand-labeled documents. Thus, Reveal_All achieves the same recall as the CAL method with true labels of hand-labeled documents.

Protocol_Label uses a strict Label-Verification protocol that requires Alice to report the labels of all hand-labeled documents correctly. For Protocol_Label, we assume that Alice truthfully reports all true labels of hand-labeled documents. With this assumption, Protocol_Label has the same labels of the hand-labeled documents in each iteration as Reveal_All. Thus, the recall of Protocol_Label is the same as that of the baseline Reveal_All. In Section 4, we show that Protocol_Label guarantees a small (training recall) loss

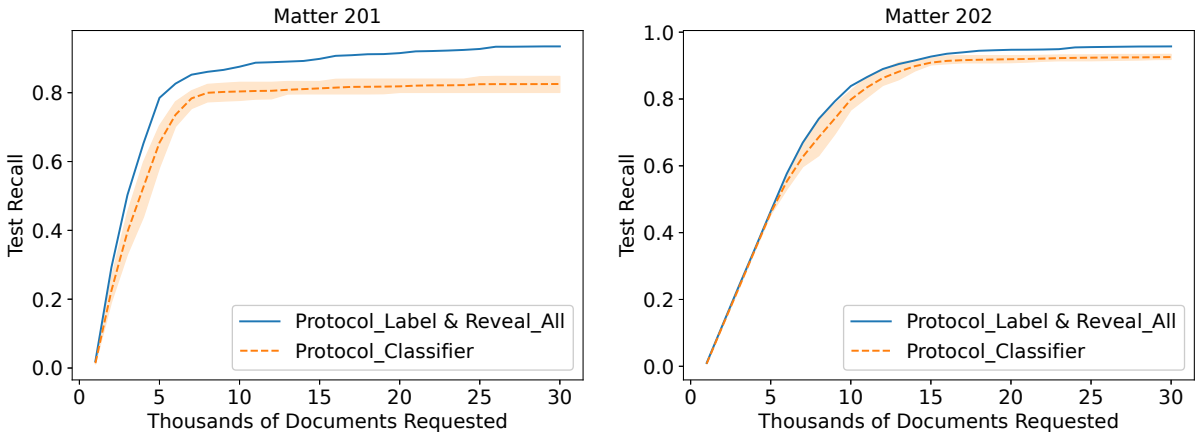

Figure 4: Test recall of Protocol_Label (CAL with the sampling protocol for label report), Protocol_Classifier (CAL with the sampling protocol for classifier report) and Reveal All (CAL by revealing all requested documents) on the datasets Matter 201 and 202. We plot the test recall of three protocols implementing the CAL method with the number of iterations $T = 1, 2, \cdots, 30$ and select $N = 1000$ new documents for review in each iteration. We repeat our protocols for 10 times and plot the average recall with the corresponding ranges.

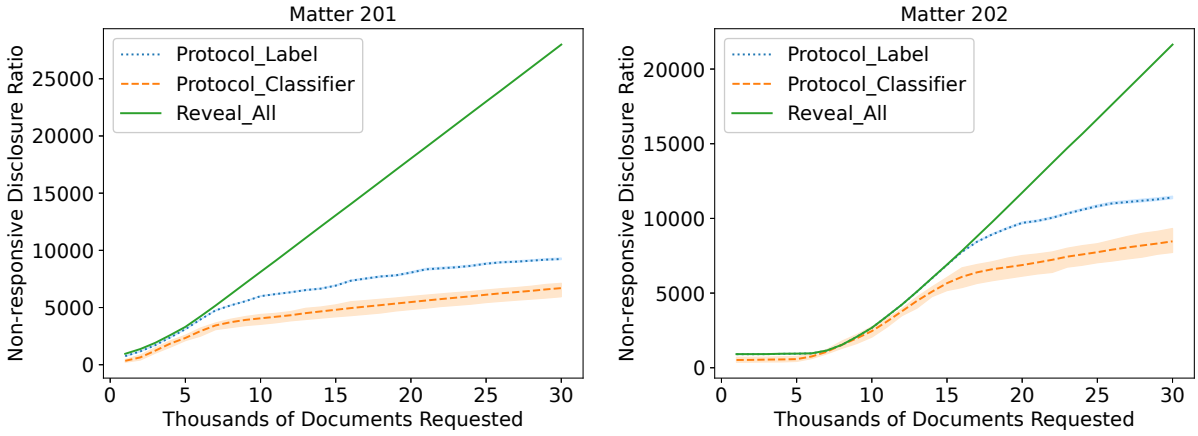

Figure 5: Non-responsive disclosure of Protocol_Label (CAL with the sampling protocol for label report), Protocol_Classifier (CAL with the sampling protocol for classifier report) and Reveal All (CAL by revealing all requested documents) on the datasets Matter 201 and 202. We plot the non-responsive disclosure of three protocols with the number of iterations $T = 1, 2, \cdots, 30$ in the CAL method that selects $N = 1000$ new documents for review in each iteration. We repeat our protocols for 10 times and plot the average non-responsive disclosure with the corresponding ranges.

in each iteration, and Alice would be partially truthful under proper assumptions. We expect that Protocol_Label will have a small test recall loss compared to the baseline in practice.

Protocol_Classifier further relaxes the requirement on the report of Alice. It uses a Label-Verification protocol that only requires Alice to report the optimal threshold on the hand-labeled documents. Figure 4 shows that when Alice truthfully reports the optimal threshold, the recall of Protocol_Classifier decreases by at most 10% compared to the recall of Reveal_All.

Figure 5 shows the non-responsive disclosure of three protocols as the number of hand-labeled documents increases. The baseline Reveal_All reveals all hand-labeled documents to Bob for verification. Thus, it has the maximum non-responsive disclosure, which equals the number of non-responsive documents in all hand-labeled documents.

Instead of revealing all hand-labeled documents, Protocol_Label uses the Label-Verification protocol for label report (Algorithm 2) to sample a subset of requested documents. The non-responsive disclosure of Protocol_Label is up to 50% lower than that of Reveal_All.

Protocol_Classifier is less strict on Alice's report and is tolerant of errors in Alice's report. It uses a Label-Verification protocol for classifier report (Algorithm 3) with small and adaptive sampling probabilities. Protocol_Classifier further decreases the non-responsive disclosure by up to 20% compared to that of Protocol_Label.

According to empirical results, when Alice truthfully reports all labels of requested documents, Protocol_Label achieves the same recall as the baseline with a much smaller non-responsive disclosure than the baseline. With the analysis in Section 4, we expect that in practice, Protocol_Label will have a small recall loss compared to the baseline with a small non-responsive disclosure. Protocol_Classifier is tolerant of errors in Alice's report and only requires Alice to report the optimal threshold. The empirical results show that when Alice truthfully reports the optimal threshold, Protocol_Classifier has at most 10% recall loss compared to the baseline with less than 1/3 of the non-responsive disclosure of the baseline.

4 THEORETICAL GUARANTEES

In this section, we describe the Label-Verification protocols used in our e-discovery protocols for the non-realizable setting. We prove theoretical guarantees of recall and non-responsive disclosure for these protocols. These guarantees are for each single call of the Label-Verification protocol. We do not have theoretical results for the combination of these protocols with the CAL method.

We consider the one-dimensional instance (R, f) as the input of our Label-Verification protocols, where the set of documents are embedded in the one-dimensional space, $R = \{x_1, x_2, \ldots, x_N\} \subset \mathbb{R}$. Note that for high-dimensional instances, our e-discovery protocols Protocol_Label and Protocol_Classifier simulate the CAL method and iteratively call these Label-Verification protocols on a subset $R \subset X$ with the highest scores according to the current linear classifier. Thus, this subset of documents R and their scores given by the current linear classifier form a one-dimensional sub-instance. Our analysis provides guarantees for each call of the Label-Verification protocol on the sub-instance (R, f).

We now define some useful notions used in our Label-Verification protocol for label report (Algorithm 2) and Label-Verification protocol for classifier report (Algorithm 3). For each document $x \in R$, let $f(x) \in \{-1, 1\}$ be the true label of this document x, and $f_A(x)$ be the label of this document x reported by Alice. We use t^* to denote the optimal threshold to classify these points with true labels, which minimizes the error

$$t^* \in \arg\min_{t \in \mathbb{R}} |\{x \in R : f(x) \neq \text{sign}(x - t)\}|.$$

If there is more than one optimal threshold, we let t^* be the largest one. Similarly, we use t_A^* to denote the optimal threshold to classify the data points with Alice's labels, i.e.

$$t_A^* \in \arg\min_{t \in \mathbb{R}} |\{x \in R : f_A(x) \neq \text{sign}(x - t)\}|.$$

If there are multiple optimal thresholds for Alice's report, then we let t_A^* be the smallest one. Let $\text{err}(t) = |\{x \in R : f(x) \neq \text{sign}(x-t)\}|$ be the error of the threshold t on the true labels. Let $\text{err}_A(t) = |\{x \in R : f_A(x) \neq \text{sign}(x - t)\}|$ be the error of the threshold t on Alice's report. For simplicity, we use $t^* = t^*(R, f)$ and $\text{err}^* = \text{err}(t^*)$ to denote the best threshold and the optimal error.

The instance (R, f) is non-realizable, which means responsive and non-responsive documents in (R, f) can not be perfectly separated by a linear classifier, i.e. the error of the best classifier $\text{err}(t^*)$ is strictly greater than 0. We consider the two different settings for Alice's report in the Label-Verification protocol. In Section 4.1, we consider the Label-Verification protocol in which Alice is requested to report all labels of documents in R correctly. In Section 4.2, we provide a robust Label-Verification protocol in which Alice is only requested to report the best classifier on (R, f).

4.1 Report Labels

We first consider the setting in which Alice is requested to report all labels of documents in R correctly. We provide a Label-Verification protocol for this strict label report setting. We show the following theorem for this protocol. The proofs in this section are deferred to Appendix A.1.

Theorem 4.1. *Given an one-dimensional instance (R, f), failure probability $\delta \in (0, 1)$ and error tolerance $k \geq 1$, Label-Verification Protocol for Label Report (Algorithm 2) satisfies*

(1) *(Recall) The recall is at least $1 - (\text{err}^* + k - 1)/N^+$ with probability at least $1 - \delta$;*
(2) *(Non-responsive disclosure) If Alice reports the true labels f on R, then the expected non-responsive disclosure is at most*

$$\left(2 + \frac{2\,\text{err}^*}{k}\right) \ln N^- \ln(1/\delta) + \text{err}^*,$$

where N is the number of data points in R, N^+, N^- are the number of positive points and negative points in R respectively, err^* is the optimal error.

Note that $\text{err}^* = \text{FN}^* + \text{FP}^*$, where FN^* and FP^* are the number of false negatives and false positives given by the optimal classifier on (R, f). The recall of the optimal classifier is $\text{REC}^* = 1 - \text{FN}^*/N^+$. For $k = 1$, Label-Verification Protocol for Label Report is guaranteed to achieve $1 - \text{err}^*/N^+ = \text{REC}^* - \text{FP}^*/N^+$ recall on (R, f) with probability as least $1-\delta$. This means, except for the false negatives in the optimal classifier, Alice can potentially hide FP^* extra positives as negatives. Alice can utilize the false positives of the optimal classifier to hide the same number of positive points as negatives. The error tolerance $k \geq 1$ captures the trade-off between the recall and non-responsive disclosure. For smaller k, this protocol has a stronger recall guarantee with a larger non-responsive disclosure for verification.

Consider a nice instance (R, f) with a constant optimal error err^*. With a small constant error tolerance k, this protocol has a small recall loss compared to the recall of the optimal classifier REC^*. With a small constant failure probability δ, this protocol has $O(\log N^-)$ non-responsive disclosure if Alice reports true labels, which is much smaller than the number of true negatives in R.

This Label-Verification protocol works as follows. It first computes the optimal classifier t_A^* for Alice's report f_A. Then, it asks Bob to verify all documents reported as positive by Alice and documents classified as positive by t_A^*. It samples the rest documents independently in decreasing order of their positions. The document with the i-th highest position is sampled and sent to Bob with probability proportional to $1/i$. If Bob disagrees with Alice on the label of this document, then this document is sent to the court to settle.

The sampling process stops if it goes through all documents reported as negative by Alice or if the protocol detects Alice made an error. If the protocol detects Alice made an error, then Trent sends all documents reported as negative by Alice to Bob to verify and sends any disagreement to the court to settle. Finally, the protocol outputs the label of each document as the court decision if this document is sent to the court; otherwise outputs the label of this document reported by Alice. In this output, all documents labeled as positive are either verified by Bob or settled by the court. Thus, documents labeled as positive in the output are always correct. The pseudo-code of our sampling protocol is shown in Algorithm 2.

Algorithm 2 Label-Verification Protocol for Label Report

Input: a set of documents R embedded in the one-dimensional space $\mathbb{R}$, error tolerance $k \geq 1$, and failure probability $\delta \in (0, 1)$
Output: labels of documents in R
Alice sends all points and their labels $\{(x, f_A(x))\}_{x \in X}$ to Trent.
Trent computes the optimal threshold t_A^* for all points with labels reported by Alice.
Trent sends all positive points reported by Alice and all points $x \in (t_A^*, +\infty)$ to Bob.
Trent goes through all points in $(-\infty, t_A^*)$ labeled as negative by Alice in decreasing order of their embedding, $t_A^* > x_1 > x_2 > \cdots > x_m$.
for each negative point x_i **do**
    Trent sends this point to Bob with probability $p_i = \min\{1, c/i\}$, where

$$c = \left(2 + \frac{2\,\mathrm{err}_A(t_A^*)}{k}\right) \ln(1/\delta).$$

    Bob labels this point and sends the label to Trent.
    If Bob labels this point as positive, then Trent sends this point to the court. If this point is a true positive, then Trent sends all points to Bob. Bob sends the labels of these points to Trent. Trent sends the points that have different labels from Alice and Bob to the court to settle.
end for
Output the label of each document $x \in R$ as $f_A(x)$ if x is not sent to the court; otherwise, output the label of this document decided by the court.

Note that the non-responsive disclosure bound in Theorem 4.1 applies when Alice reports the true labels f. We now examine the best response of Alice to this protocol. To do this, we first specify the loss function of Alice L_A. Note that Alice will benefit from the responsive documents that are not retrieved and sent to Bob. Alice also has a privacy loss due to the disclosure of any non-responsive documents to Bob. Thus, Alice has a loss function L_A on the outcome $B \subset R$ of the protocol that depends on the recall $\mathrm{REC}(B)$ and the non-responsive disclosure $\mathrm{NRD}(B)$. We assume the loss function of Alice is linear, $L_A(B) = \mathrm{NRD}(B) + c \cdot N^+ \cdot (\mathrm{REC}(B) - 1)$.

By assuming Alice's loss function L_A is linear and the instance (R, f) has enough number of true negatives, we have the following theorem on the best response of Alice. The proof is deferred to Appendix A.1.

Theorem 4.2. *Suppose Alice has a linear loss function on the outcome $B \subset R$, $L_A(B) = \mathrm{NRD}(B) + c \cdot N^+ \cdot (\mathrm{REC}(B) - 1)$ for some constant $c \geq 0$. For any $\delta \in (0, 1/3)$ and $k \geq 1$, consider an instance (R, f) with the number of negative points at least*

$$N^- > \max\{3(2 + 2\mathrm{err}^*/k) \cdot \ln(1/\delta) \cdot \ln N^- + 3\mathrm{err}^*, c \cdot N^+\}.$$

Alice's best response to Label-Verification Protocol with Label Report satisfies that

(1) *(Recall) The recall is always at least $1 - (\mathrm{err}^* + k)/N^+$;*
(2) *(Non-responsive disclosure) The expected non-responsive disclosure is at most $(2 + 2\mathrm{err}^*/k) \ln N^- \ln(1/\delta) + (c + 1)\mathrm{err}^* + ck$.*

Particularly, for $k = 1$, Alice's best response truthfully reports the labels of all points classified as positive by the optimal classifier of (R, f), which provides the recall at least REC^*.

If Alice does not report the true labels, then the outcome of our protocol can be divided into two cases: (1) If the protocol detects a positive point hidden by Alice, then it reveals all documents to Bob for verification. In this case, the recall is 1 and non-responsive disclosure is the number of true negatives N^-. (2) If the protocol does not detect any hidden positive, then the bounds for recall and non-responsive disclosure in Theorem 4.1 hold. By assuming enough true negatives N^- and Alice's loss function, the loss in the first case is larger than that of the second case. Then, Alice will report the true labels for a large portion of documents in the best response. However, if some false positives of the optimal classifier are far away from the classifier boundary, which are outliers, then Alice may hide these positives as negative with a very small probability of being detected. Hence, we only partially characterize Alice's best response.

4.2 Report a Classifier

We now consider a setting where Alice is only requested to report a good linear classifier on R instead of labeling all documents correctly. This setting is tolerant of errors made by Alice since Alice is requested to find a good linear classifier and is allowed to make errors. We provide an error-tolerant Label-Verification protocol for this classifier report setting. We show the following theorem for this Label-Verification protocol (Algorithm 3). The proof of this theorem is in Appendix A.2

Theorem 4.3. *Given an one dimensional instance (R, f) and failure probability $\delta \in (0, 1)$, Label-Verification Protocol for Classifier Report (Algorithm 3) satisfies*

(1) *(Recall) The recall is at least $1 - 3\mathrm{err}^*/N^+$ with probability at least $1 - \delta$;*
(2) *(Non-responsive disclosure) If Alice reports an optimal classifier for (R, f), then the expected non-responsive disclosure is at most*

$$2\,\mathrm{err}^* \ln N \ln(N/\delta) + \mathrm{err}^*,$$

where N is the number of data points, N^+ is the number of positive points, and err^* is the optimal error for this instance.

The Label-Verification protocol for classifier report is guaranteed to achieve $1-3\mathrm{err}^*/N^+$ recall on (R, f) with probability as least $1-\delta$. If the classifier reported by Alice has more than $3\mathrm{err}^*$ classification errors on (R, f), then the protocol detects this classifier reported by Alice is not optimal with probability at least $1 - \delta$. If the protocol

Algorithm 3 Label-Verification Protocol for Classifier Report

Input: a set of N documents R embedded in the one-dimensional space $\mathbb{R}$ and failure probability $\delta \in (0, 1)$
Output: labels of documents in R
Alice sends all points X and a classifier t_A to Trent.
Trent asks Alice to label all points in $[t_A, +\infty)$. Trent sends all points in $[t_A, +\infty)$ to Bob. Bob sends the labels of these points to Trent. Trent sends the points that have different labels from Alice and Bob to the court to settle.
Trent goes through all points in $(-\infty, t_A)$ in decreasing order, $t_A > x_1 > x_2 > \cdots > x_m$.
Set the counts of positives and negatives $M^+ = M^- = 0$ and the weight $W = 1$.
for each point x_i **do**
    Trent sends this point to Bob with probability $p_i = \min\{1, c/W\}$, where
$$c = 2\ln(N/\delta).$$
    Bob labels this point and sends the label to Trent.
    If Trent does not send this point to Bob or Bob labels this point as negative, then $M^- = M^- + 1$ and $W = W + 1$.
    If Bob labels this point as positive, then Trent sends this point to the court. If this point is a true positive, then $M^+ = M^+ + 1$ and $W = 1$.
    If $M^+ > M^-$, then Trent sends all points in $(-\infty, t_A)$ to Bob. Bob sends the labels of these points to Trent. Trent sends the points labeled as positive by Bob to the court to settle.
end for
Output the labels of documents in R. If the document x is not sent to the court, then label x by Alice's report for $x \geq t_A$ and label x as negative for $x < t_A$; otherwise, output the label of this document decided by the court.

detects this classifier is not optimal, then it reveals all documents in R to Bob for verification. For nice instances with constant optimal error err^*, with a constant failure probability δ, this protocol has $O(\log^2 N)$ non-responsive disclosure if Alice reports the optimal classifier for (R, f). If the number of negative points $N^- \gg N^+$, then the non-responsive disclosure of this protocol is much smaller than the number of all negative points.

The Label-Verification Protocol for Classifier Report is similar to the Label-Verification Protocol for Label Report. The main difference is that the stopping condition for Label-Verification Protocol for Classifier Report (Algorithm 3) is when the classifier t_A reported by Alice is detected to be not optimal instead of detecting one error made by Alice in Label-Verification Protocol for Label Report (Algorithm 2). The classifier reported by Alice is detected to be not optimal if and only if enough positives in $(-\infty, t_A)$ are detected and a better classifier with smaller errors is already found. Since the stopping condition is changed, this new protocol might allow multiple errors made by Alice. Therefore, the selection rule for documents revealed to Bob is changed to have multiple epochs accordingly. In each epoch, the document with the i-th highest position in this epoch is sampled with probability proportional to $1/i$. Whenever the protocol detects an error made by Alice, the protocol starts a new epoch and resets the sampling probability to 1. In this protocol, Alice is only asked to label points in $[t_A, +\infty)$ not all points. We set $f_A(x)$ to be Alice's report if $x \geq t_A$, and $f_A(x) = -1$ if $x < t_A$. Finally, the protocol outputs the label of each document x as the court decision if this document is sent to the court; otherwise, label this document x by $f_A(x)$. In this output, all documents labeled as positive are correct since they are either verified by Bob or settled by the court.

4.3 Lower Bound

In this section, we provide a lower bound on the non-responsive disclosure of protocols with a large recall.

Theorem 4.4. *For any multi-party binary classification protocol with the recall at least* $\mathrm{REC}^*_{(R,f)}$ *on all one-dimensional instances* (R, f)*, the worst-case non-responsive disclosure of this protocol is at least* $\Omega(\ln N)$ *on the instance with* N *documents, where* $\mathrm{REC}^*_{(R,f)} = 1 - \mathrm{FN}^*_{(R,f)}/N^+_{(R,f)}$ *is the recall given by the optimal classifier on* (R, f).

This lower bound implies that to achieve a recall of at least REC^* on all one-dimensional instances, the $\log N$ dependence on the number of documents N in non-responsive disclosure is necessary. For instances with constant err^*, with a constant failure probability δ, our Label-Verification Protocol for Label Report has $O(\log N)$ non-responsive disclosure if Alice reports true labels, which matches this worst-case lower bound. For instances with constant err^* and constant δ, our Label-Verification Protocol for Classifier Report has $O(\log^2 N)$ non-responsive disclosure if Alice reports the optimal classifier.

Proof of Theorem 4.4. We construct a family of $\log_2 N$ one-dimensional instances as follows. These instances have the same data point positions $R \subseteq \mathbb{R}$ with $|R| = N$ data points $x_1 \geq x_2 \geq \cdots \geq x_N$. These $\log_2 N$ instances have different labels f_j for $j = 1, \cdots, \log_2 N$. We partition the data points into $\log_2 N$ buckets $B_0 = \{x_1\}$ and $B_j = \{x_i : 2^{j-1} + 1 \leq i \leq 2^j\}$ for $j = 1, 2, \cdots, \log_2 N$. The label function f_j assigns data points in $B_0 \cup B_j$ as positive points and the rest points as negative points. These $\log_2 N$ instances have different best linear classifiers. The best threshold for the instance (R, f_j) is $t^*_j = x_i$ where $i = 2^j$. If the protocol can not distinguish these $\log_2 N$ instances, then the recall of the protocol will be smaller than $1 - (\mathrm{FN}^*_{(R,f_j)} + 1)/N^+_{(R,f_j)}$ for one of these instances (R, f_j). Thus, to distinguish these $\log_2 N$ instances, the protocol needs to choose one point from each bucket. Then, the non-responsive disclosure is at least $\log_2 N$. □

5 CAN WE HAVE MUCH SMALLER PRIVACY LOSS?

In this section, we make an empirical comparison between the non-responsive disclosure of our protocols from Section 3 with that of the critical points protocol proposed in Dong et al. [2022]. Note that in the realizable case, the critical points protocol has minimal non-responsive disclosure. Based on this empirical comparison, we conjecture that the non-responsive disclosure of our protocols can not be improved significantly.

We observe in Figure 6 that although the non-responsive ratio of the critical points protocol is lower (7% vs 20%) than that of

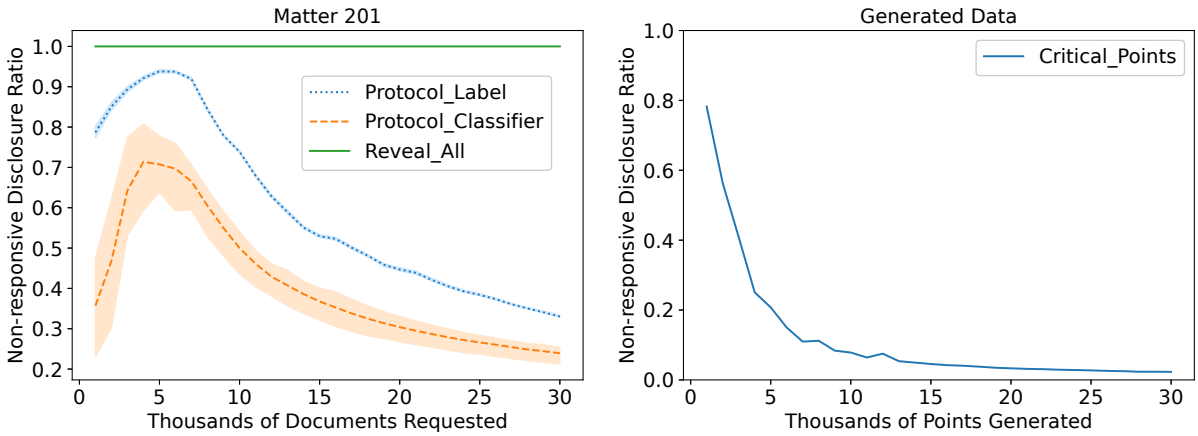

Figure 6: Non-responsive disclosure ratio (the non-responsive disclosure divided by the number of non-responsive documents in the hand-labeled documents) of our protocols on the review task Matter 201 (left panel) and Non-responsive disclosure ratio (the non-responsive disclosure divided by the number of non-responsive points in the generated points) of the critical point protocol on the simulated data (right panel). For the right panel, the data are generated by first sampling from two standard Gaussian and then shifting misclassified points such that the dataset is realizable. The ratio of the number of points in the two types is 0.05, which is roughly the fraction of responsive documents in Matter 201. Algorithm 4 was used to compute the number of critical points, which is the non-responsive disclosure.

our protocol, the asymptotic trend of the non-responsive ratio of our protocols and critical points protocol are quite similar. Note that the critical points protocol has the minimum non-responsive disclosure in the realizable setting. Even in the realizable case, there is a significant and unavoidable amount of non-responsive disclosure. Since the learning problem in the non-realizable setting is much harder than in the realizable setting, we conjecture that the non-responsive disclosure of our protocols is unlikely to have significant room for improvement.

We note that the experiment is more suggestive than rigorous. There are several significant differences between the left and right panels:

(1) The data distributions are different. The left panel uses real data and is non-realizable, while the dataset used in the right panel is generated by using the mixture of two Gaussians and enforcing the sampled points to be realizable. (The enforcing step shifts the data points that are misclassified by the optimal classifier such that the dataset is realizable. We do not use this enforcing step on the real data used in the left panel because the dimension of this real data exceeds the computational limitation of the critical points protocol.)
(2) The dimensions of the two datasets are different. The real-world data used in the left panel has dimensions in hundreds of thousands while the generated data in the right panel has only 100 dimensions. This is more of a computational difficulty: even with our significant improvement, the critical point protocol is still computationally demanding.

In the rest of the section, we overview a fast implementation of the critical points protocol with the details in Appendix B and hopefully shine new thoughts on further improvements. We first introduce some terms and notations that are useful in this section. Recall that an instance (X, f) consists of a set of documents $X \subseteq \mathbb{R}^d$ and $f : X \to \{-1, 1\}$ indicates the labels of documents. Let $n^+ = |\{x \in X : f(x) = 1\}|$ be the number of responsive documents in X and $n^- = |\{x \in X : f(x) = -1\}|$ be the number of non-responsive documents.

The critical points protocol reveals all positives and critical points in negatives reported by Alice. A document reported as negative by Alice is a critical point if and only if after flipping the label of this document to positive, the labels reported by Alice are still linearly separable. Thus, critical points are those negative points in the boundary of all reported negatives, which serves as a proof for the correctness.

The algorithm provided in Dong et al. [2022] requires solving n linear programs, each of size n by d. Assuming the best possible solver for the dense linear program (which is not far in reach, see van den Brand et al. [2020]), this takes $O(n^2 d)$ time and is too slow in practice. We provide a fast implementation of critical points protocol by using the algorithm for finding extremal points proposed by Clarkson [1994]. Specifically, if the number of critical points is ℓ, then this fast implementation takes $O(nd\ell)$ time, which is much faster than that in Dong et al. [2022] since the number of critical points ℓ is usually much smaller than n. The details of this implementation are discussed in Appendix B.

6 CONCLUSION

In this work, we explore accountability concerns in multi-party classification problems that arise in e-discovery. Our protocol studies the e-discovery protocol setting in Dong et al. [2022] and gives a new protocol that goes beyond the realizable case. Experiments on real and simulated data show that the three desiderata can be simultaneously achieved: (1) accountability (2) high recall and (3)

low non-responsive disclosure. We further prove that high recall is guaranteed even when the defendant (Alice) behaves strategically, but only in the single-dimensional case.

There are many interesting directions for future research. A natural question that is left open by this work is whether we can obtain theoretical guarantees in higher dimensions (analogous to Theorems 4.1 and 4.3). This may require additional assumptions about the data distribution to overcome the computational intractability of the non-realizable linear classification problem in high dimensions, even without any accountability consideration. Another interesting question is to consider the different levels of privacy significance among documents, which is common in legal practice. It requires to design a more delicate measurement of privacy loss. In this work, we use the non-responsive disclosure as a simplified measurement of privacy loss. Finally, it is important to further consider the objectives of the defendant and plaintiff and fully characterize the best response strategy of the defendant (Alice) under natural assumptions. It is also interesting to consider different compelling motions when the protocol detects enough errors in Alice's report instead of just revealing all requested documents.

ACKNOWLEDGMENTS

This work is supported by NSF award CCF 1934931 and ECCS 2216970. Thank the anonymous referees for their valuable comments and helpful suggestions.

REFERENCES

Pranjal Awasthi, Maria Florina Balcan, and Philip M. Long. 2017. The Power of Localization for Efficiently Learning Linear Separators with Noise. *J. ACM* 63, 6, Article 50 (jan 2017), 27 pages. https://doi.org/10.1145/3006384

Wei-Cheng Chang, Felix X Yu, Yin-Wen Chang, Yiming Yang, and Sanjiv Kumar. 2020. Pre-training tasks for embedding-based large-scale retrieval. *arXiv preprint arXiv:2002.03932* (2020).

Kenneth L Clarkson. 1994. More output-sensitive geometric algorithms. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*. IEEE, 695–702.

Gordon V Cormack and Maura R Grossman. 2014. Evaluation of machine-learning protocols for technology-assisted review in electronic discovery. In *Proceedings of the 37th international ACM SIGIR conference on Research & development in information retrieval*. 153–162.

Gordon V Cormack and Maura R Grossman. 2017. Technology-Assisted Review in Empirical Medicine: Waterloo Participation in CLEF eHealth 2017. *CLEF (working notes)* 11 (2017).

Gordon V Cormack and Mona Mojdeh. 2009. Machine Learning for Information Retrieval: TREC 2009 Web, Relevance Feedback and Legal Tracks.. In *TREC*.

Jinshuo Dong, Jason Hartline, and Aravindan Vijayaraghavan. 2022. Classification Protocols with Minimal Disclosure. In *Proceedings of the 2022 Symposium on Computer Science and Law*. 67–76.

Brown v. *Tellermate Holdings Ltd.* 2014. *Case No. 2:11-cv-1122, 2014 U.S. Dist. LEXIS 90123* (2014). https://casetext.com/case/brown-v-tellermate-holdings-ltd

Hyles v. *New York City*. 2016. *10 Civ. 3119 (AT)(AJP)* (2016). https://casetext.com/case/hyles-v-nyc

Moore v. *Groupe*. 2012. *868 F. Supp. 2d 137* (2012). https://casetext.com/case/moore-v-groupe

O Goldreich, S Micali, and A Wigderson. 1987. How to play ANY mental game. In *Proceedings of the nineteenth annual ACM symposium on Theory of computing*. 218–229.

Shafi Goldwasser, Silvio Micali, and Charles Rackoff. 1989. The Knowledge Complexity of Interactive Proof Systems. *SIAM J. Comput.* 18, 1 (1989), 186–208.

Shafi Goldwasser, Guy N. Rothblum, Jonathan Shafer, and Amir Yehudayoff. 2021. Interactive Proofs for Verifying Machine Learning. In *12th Innovations in Theoretical Computer Science Conference (ITCS 2021) (Leibniz International Proceedings in Informatics (LIPIcs), Vol. 185)*, James R. Lee (Ed.). Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 41:1–41:19. https://doi.org/10.4230/LIPIcs.ITCS.2021.41

Maura R Grossman and Gordon V Cormack. 2010. Technology-assisted review in e-discovery can be more effective and more efficient than exhaustive manual review. *Rich. JL & Tech.* 17 (2010), 1.

Maura R Grossman and Gordon V Cormak. 2012. Inconsistent responsiveness determination in document review: Difference of opinion or human error. *Pace L. Rev.* 32 (2012), 267.

Venkatesan Guruswami and Prasad Raghavendra. 2009. Hardness of Learning Halfspaces with Noise. *SIAM J. Comput.* 39, 2 (2009), 742–765. https://doi.org/10.1137/070685798 arXiv:https://doi.org/10.1137/070685798

Bruce Hedin, Stephen Tomlinson, Jason R Baron, and Douglas W Oard. 2009. Overview of the TREC 2009 Legal Track.. In *TREC*.

Adam Tauman Kalai, Adam R. Klivans, Yishay Mansour, and Rocco A. Servedio. 2008. Agnostically Learning Halfspaces. *SIAM J. Comput.* 37, 6 (2008), 1777–1805. https://doi.org/10.1137/060649057 arXiv:https://doi.org/10.1137/060649057

Michael J. Kearns, Robert E. Schapire, and Linda M. Sellie. 1992. Toward Efficient Agnostic Learning. In *Proceedings of the Fifth Annual Workshop on Computational Learning Theory* (Pittsburgh, Pennsylvania, USA) *(COLT '92)*. Association for Computing Machinery, New York, NY, USA, 341–352. https://doi.org/10.1145/130385.130424

Michael J. Kearns and Umesh V. Vazirani. 1994. *An Introduction to Computational Learning Theory*. MIT Press, Cambridge, MA, USA.

Daniel N Kluttz and Deirdre K Mulligan. 2019. Automated decision support technologies and the legal profession. *Berkeley Technology Law Journal* 34, 3 (2019), 853–890.

Antoine Louis and Gerasimos Spanakis. 2021. A statutory article retrieval dataset in French. *arXiv preprint arXiv:2108.11792* (2021).

Alison O'Mara-Eves, James Thomas, John McNaught, Makoto Miwa, and Sophia Ananiadou. 2015. Using text mining for study identification in systematic reviews: a systematic review of current approaches. *Systematic reviews* 4, 1 (2015), 1–22.

Allyson Haynes Stuart. 2021. A Right to Privacy for Modern Discovery. *Geo. Mason L. Rev.* 29 (2021), 675.

Jan van den Brand, Yin Tat Lee, Aaron Sidford, and Zhao Song. 2020. Solving tall dense linear programs in nearly linear time. In *Proceedings of the 52nd Annual ACM SIGACT Symposium on Theory of Computing*. 775–788.

Jie Zou and Evangelos Kanoulas. 2020. Towards question-based high-recall information retrieval: Locating the last few relevant documents for technology-assisted reviews. *ACM Transactions on Information Systems (TOIS)* 38, 3 (2020), 1–35.